SMART PARKING SYSTEM

Micro Project Report

Submitted to the APJ Abdul Kalam University, Kerala
for the partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology
in
Computer Science and Engineering

By

KRISHNANUNNI UR-PRN25CS068
MUHAMMED N-PRN25CS078
NAVNIT M-PRN25CS089
SANJAY KRISHNA S-PRN25CS106

Guided by
Prof. ANISH A AZIZ

During
OCTOBER 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING PERUMON
CAPE - GOVT. OF KERALA, KOLLAM(D), KERALA

Table of Contents

1.INTRODUCTION 2

2.SCOPE OF THE MICRO PROJECT 3

3.TECHNOLOGIES USED 4

4.PROBLEM STATEMENT 5

5.LITERATURE REVIEW / BACKGROUND 6

6.SYSTEM ANALYSIS & DESIGN 7

7.IMPLIMENTATION 7

8.DETAILS OF JAVA CLASSES AND ARCHITECTURE 8

9.RESULTS AND DISCUSSIONS 9

10.CONCLUSION 11

REFFERENCE 12

1.INTRODUCTION

Parking is an important part of travelling to workplaces, shops, railway stations, hospitals, and other public places. When drivers do not know where parking is available, they may spend extra time searching for a space. This can cause inconvenience for drivers and contribute to traffic congestion, especially in busy areas. A simple system that presents parking information and records bookings can help make the process more organized.

The **Smart Parking System** is a Java desktop application developed as a micro-project to demonstrate a basic digital parking workflow. The application provides a graphical interface through which a user can view sample parking facility information, select a parking slot, enter vehicle and duration details, and attempt to save a booking. It also includes a screen for checking the details of an existing booking. The project combines Java programming, graphical user interface design, event handling, and database connectivity.

The application is built using **Java Swing**. Its screens use components such as windows, panels, labels, buttons, and text fields. The user begins at the home screen, which provides options to find a parking spot or check parking status. Choosing the parking search option opens a screen that displays two example parking facilities in Kollam, along with sample information about their locations, available slots, and hourly prices. The user can enter a location and press the search button. In the current version, this action displays a message confirming the entered search; the location does not yet filter the sample facilities.

From the parking search screen, the user can open the slot selection screen. This screen displays twenty slots arranged in a grid, with labels from A1 to D5. Four slots are disabled as examples of occupied spaces. A user can select one of the enabled slots, and the selected slot is highlighted on the screen. Continuing opens the booking confirmation screen, where the selected slot is shown and the user can enter a vehicle number and the number of parking hours.

The booking screen checks that the vehicle number has been entered and that the parking duration is a positive number. It then calculates the fee using a fixed rate of Rs. 50 per hour. The application uses **JDBC** to connect to a **MySQL** database and insert the vehicle number, selected slot, duration, and total fee into a booking table. The parking status screen is intended to retrieve booking details using a vehicle number or booking ID.

The program is organized into separate Java classes for its screens. **HomeFrame, FindParkingFrame, ParkingSlotFrame, BookingConfirmationFrame,** and **ParkingStatusFrame** each handle a different part of the interface and workflow. The **DBConnection** class is a separate helper that provides the connection to MySQL; it is not a frame or screen. This separation makes the project easier to understand and provides a foundation for extending its features.

The current application is a prototype for learning and demonstration. Parking locations, displayed prices, available-slot counts, and occupied slots are examples defined in the source code. The program does not yet manage live availability or update a slot's status after a booking. Even with these limitations, the project demonstrates how a desktop application can guide a user through a booking process and connect a Java interface to a relational database. It can be extended in the future by storing parking facilities and slots in the database, adding live availability checks, supporting cancellations, and improving booking management.

2.SCOPE OF THE MICRO PROJECT

The scope of the Smart Parking System micro-project is to develop a basic desktop application that demonstrates how a user can move through the main steps of a parking booking process. The system is designed as an academic prototype using Java Swing for the interface and MySQL for storing booking information. Its purpose is to demonstrate a practical application of Java programming, event handling, database connectivity, and the organization of a program into separate classes.

The application provides a home screen with two main options: **Find Parking Spot** and **My Parking Status**. The parking search screen displays two sample parking facilities, their locations, example slot availability, and hourly prices. The user can enter a location and press the search button. In the current implementation, the program checks that the search field is not empty and displays a message. It does not yet retrieve facilities from the database or filter them based on the location entered.

The slot selection screen presents twenty parking slots arranged in a four-row, five-column grid. The slots are labelled from A1 to D5. Four slots—A3, B2, C4, and D1—are disabled to represent occupied spaces. The user can select an enabled slot and continue to the booking confirmation screen. The booking screen accepts a vehicle number and the number of parking hours, validates these values, calculates a fee, and attempts to save the details to MySQL.

The status screen is intended to let the user search for a booking using a vehicle number or a numeric booking ID. When a matching record is retrieved, the program displays information such as the booking ID, vehicle number, parking slot, duration, and total fee.

The project is limited to the features present in the Java source code. Parking facilities and slot states are currently examples written directly in the program. Slot availability is not updated in the database after a booking, and the displayed sample location prices are not used by the booking calculation. The booking screen currently applies a fixed rate of Rs. 50 per hour. The project does not include user accounts, online payments, booking cancellation, or live slot updates. These could be considered future extensions.

3.TECHNOLOGIES USED

The project is developed using **Java**, a programming language commonly used to build desktop, web, and mobile applications. The project uses Java classes and methods to organize the screens and their behaviour. Each major screen is implemented in a separate class, which helps keep the interface and actions for each part of the application together.

The graphical user interface is created using **Java Swing**. Swing provides components for building desktop windows and collecting user input. The project uses **JFrame** for application windows, **JPanel** for grouping interface elements, **JLabel** for displaying text, **JButton** for actions, and **JTextField** for accepting input. **JOptionPane** is used to display messages such as validation prompts, booking confirmations, and error details.

The project also uses layout managers from **Java AWT** to arrange components. **BorderLayout** organizes content into window regions, **GridLayout** arranges components in rows and columns, and **FlowLayout** places components in a line. These layout managers help position screen elements without assigning fixed coordinates to every control.

The application uses **MySQL** as its relational database management system. The database is named **smart_parking**, according to the connection URL in DBConnection.java. The Java Database Connectivity API, known as **JDBC**, is used to communicate with MySQL. The project uses **DriverManager** to establish the connection, Connection to represent an active database connection, **PreparedStatement** to execute parameterized SQL statements, and **ResultSet** to read query results.

The booking operation uses an INSERT statement to store the vehicle number, slot number, parking hours, and total fee. The status screen uses SELECT statements to search for booking records. Before the application can perform these operations, a MySQL server, a compatible MySQL JDBC driver, the **smart_parking** database, and the required bookings table must be available.

4.PROBLEM STATEMENT

Drivers who visit busy locations may find it difficult to identify a parking facility, understand its available spaces, and keep track of their booking details. When parking information is not presented clearly, users may spend additional time looking for a suitable space. A simple computer-based application can help organize parking information and record booking details in a structured way.

The problem addressed by this micro-project is the need for a basic interface that demonstrates how a user can view sample parking information, select a slot, enter vehicle and duration details, and store a booking record. The project also includes a screen for looking up booking information using a vehicle number or booking ID. It connects the user-facing interface to a relational database so that booking details can be saved and retrieved.

The application accepts a location search term, a selected parking slot, a vehicle number, parking hours, and a search value for booking lookup. The intended booking output includes a unique booking ID, the vehicle number, the selected slot, the duration, and the total fee. The fee is calculated based on the number of hours entered and a fixed rate of Rs. 50 per hour in the current implementation.

The system is divided into different screens to represent the steps in the workflow. The home screen provides navigation, the search screen displays sample parking facilities, the slot screen allows selection from the available sample slots, the booking screen collects details and saves them, and the status screen is designed to retrieve a booking.

The current system is a prototype, so it does not solve every aspect of real parking management. The facility and slot information is predefined, location searching does not filter the listings, and availability is not updated after a booking. The problem statement for this project is therefore focused on demonstrating the basic user interface and database workflow, while identifying the areas that would need further development for a live system.

5.LITERATURE REVIEW / BACKGROUND

A parking management application presents information and options through a user interface, then guides the user through tasks such as viewing facilities, selecting a space, and entering booking details. This project demonstrates that approach through a sequence of Java desktop screens.

The application's interface is built with **Java Swing**, which provides components such as windows, panels, labels, buttons, and text fields. Layout managers arrange these components on each screen. The project uses separate frames for different stages of the user journey. The **Home Frame** acts as the main menu. The **Find Parking Frame** shows sample parking facilities and a location field. The **Parking Slot Frame** displays a grid of slots and indicates which sample slots are occupied. The **Booking Confirmation Frame** accepts the vehicle number and parking duration. The **Parking Status Frame** provides a screen for looking up booking details.

The frames respond to user actions through event listeners. For example, clicking a button can open another frame, select a slot, check whether required information has been entered, or display a message. Keeping each screen in a separate class helps organize the interface and makes it easier to identify the purpose of each part of the program.

This project uses a simple database connection to support booking operations. The **DBConnection** class provides the connection used by the application, while the booking and status frames contain the code that attempts to save or retrieve booking information. Database connectivity is a supporting feature of the project; the main focus of the interface is the design and interaction of its frames.

The current version is a prototype. The parking facilities and slot states are sample values defined in the program. A future version could retrieve facility and slot information from a database, update availability after a booking, and show search results that match the location entered by the user.

6.SYSTEM ANALYSIS & DESIGN

The system is organized into two main parts: the graphical interface and the database connection. The graphical interface consists of five screens: the home screen, parking search screen, slot selection screen, booking confirmation screen, and parking status screen. The **DBConnection** class provides a connection to the MySQL database and is a helper class, not a screen.

When the application starts, **HomeFrame** displays the main menu. If the user selects **Find Parking Spot**, the application opens **FindParkingFrame**. This screen displays sample parking location cards and a search field. The **View Details** button on a card opens **ParkingSlotFrame**, which presents the parking slot grid. The user selects an enabled slot and presses **Continue** to open **BookingConfirmationFrame**.

In the booking screen, the selected slot is displayed. The user enters a vehicle number and the number of parking hours. The application checks the input and calculates the fee using the current fixed rate. If the database connection and table are available, the application inserts the booking details into MySQL. From the home screen, the user can also open **ParkingStatusFrame** and attempt to retrieve a booking using a vehicle number or booking ID.

7.IMPLIMENTATION

The application is implemented in Java using Swing for the graphical interface and JDBC for the database operations. The work is divided among separate classes so each screen has its own layout and event handling.

HomeFrame is the starting screen. It creates the application window and displays buttons for finding parking and checking parking status. The button listeners create the corresponding screen objects. **FindParkingFrame** displays a location search field and two sample parking facility panels. The search action checks whether the field is empty. If a location has been entered, it displays a message, but it does not currently change or filter the parking facility panels.

ParkingSlotFrame builds a grid containing twenty slots, from A1 through D5. The program disables A3, B2, C4, and D1 to represent occupied slots. When the user clicks an enabled slot, that button is highlighted and the selected slot name is displayed. The **Continue** button checks whether a slot has been selected before opening the booking screen.

BookingConfirmationFrame receives the selected slot as a constructor argument. It displays that slot and accepts a vehicle number and parking duration. The program checks that the vehicle number is not blank and that the duration is a positive integer. It multiplies the entered hours by Rs. 50 to calculate the fee. It then prepares an INSERT statement and supplies the booking values using a **PreparedStatement**.

ParkingStatusFrame allows the user to enter a vehicle number or booking ID. It first attempts a vehicle number search and then attempts an ID search when the input is numeric and no vehicle record is found. The booking fields are displayed in labels when a record is retrieved.

DBConnection contains the JDBC connection method for the local MySQL database. The project requires a configured database and bookings table, as well as a suitable MySQL JDBC driver.

8.DETAILS OF JAVA CLASSES AND ARCHITECTURE

The system uses five Java classes for the graphical interface and one helper class for connecting to the database. Each interface class extends **JFrame**, which allows it to create a separate application window. This arrangement divides the user workflow into manageable screens.

HomeFrame creates the starting window and displays the application title and navigation options. It provides access to the parking search and status screens. Its main method launches the application by creating a **HomeFrame** object.

FindParkingFrame displays a location input field, a search button, and two sample parking facility panels. Each panel shows a name, location, number of available slots, and hourly price. The search button checks for empty input and displays a message. The facility information is defined in the code rather than retrieved from MySQL. The **View, Details** button opens the slot selection screen.

ParkingSlotFrame creates the slot selection interface. It generates slot labels from A1 to D5 using nested loops. Four slots are disabled as sample occupied slots. When an enabled button is clicked, it is highlighted and the selected-slot label is updated. The **Continue** button prevents the user from proceeding until a slot has been selected.

BookingConfirmationFrame receives the selected slot in its constructor. It collects a vehicle number and parking hours, validates the information, calculates the total fee, and attempts to insert the record into the database. It uses a parameterized SQL statement to provide the values to MySQL.

ParkingStatusFrame presents a search field and labels for displaying booking information. It searches by vehicle number and then tries a booking ID search for numeric input. It uses a **ResultSet** to read the returned booking fields.

DBConnection provides the **getConnection()** method. It uses **DriverManager** to connect to the MySQL database identified in the JDBC URL. It does not create a window or represent a GUI frame.

The screen flow is: **HomeFrame → FindParkingFrame → ParkingSlotFrame → BookingConfirmationFrame**. Status lookup follows a separate flow: **HomeFrame → ParkingStatusFrame**. The booking and status screens use **DBConnection** to access the database.

9.RESULTS AND DISCUSSIONS

The Smart Parking System provides a sequence of graphical screens that demonstrates how a user can explore sample parking information and select a parking slot. The frames are built using Java Swing components such as labels, buttons, text fields, and panels. Each frame has a specific purpose, and the buttons allow the user to move through the application.

The **Home Frame** is the first screen displayed when the program starts. It shows the title “Smart Parking System” and two main options: **Find Parking Spot** and **My Parking Status**. Selecting either button opens the corresponding screen. This gives the user a clear starting point for using the application.

The **Find Parking Frame** contains a location search field and displays two example parking facilities: City Parking Centre and Railway Parking Area, both shown in Kollam. Each facility panel displays a sample number of available slots and an hourly price. The user can enter a location and press **Search**. In the current version, the program checks that the field is not empty and displays a message; it does not filter the displayed facilities based on the entered location. The **View Details** button opens the slot selection screen.

The **Parking Slot Frame** displays twenty slots in a grid, labelled from A1 to D5. Four slots—A3, B2, C4, and D1—are disabled to represent occupied spaces. The user can select an enabled slot, which is highlighted, and the selected slot name appears on the screen. If the user presses **Continue** without selecting a slot, the program displays a prompt. After a selection, the application opens the booking confirmation screen and passes the selected slot to it.

The **Booking Confirmation Frame** displays the selected slot and provides fields for entering a vehicle number and parking duration. It checks that the vehicle number is not blank and that the hours are a valid positive number. The frame calculates the fee using the fixed rate of Rs. 50 per hour and displays a confirmation or error message. This part of the application demonstrates how user input can be checked and processed before a booking is recorded.

The **Parking Status Frame** provides a search field where a user can enter a vehicle number or booking ID. It is designed to display the booking ID, vehicle number, parking slot, number of hours, and total fee. The screen also provides a message when the search field is empty or when no matching booking is found. The lookup should be tested with a saved booking to confirm that the details appear correctly.

The application uses a separate **DBConnection** class to connect to the local MySQL database. The booking frame attempts to save booking details, and the status frame attempts to retrieve them. MySQL connectivity requires the database and table to be configured separately.

Overall, the frames demonstrate a clear navigation flow from the home screen to parking search, slot selection, and booking. They also show how a desktop application can guide users through a task using separate screens. The facility information, prices, available-slot counts, and occupied slots are currently sample values written in the program. The location search does not filter results, slot availability is not updated after a booking, and the booking calculation uses one fixed hourly rate. These are limitations of the current version and possible areas for future improvement.

10.CONCLUSION

The Smart Parking System micro-project demonstrates how a Java desktop application can guide a user through a simple parking search and booking process. It uses separate graphical frames to present each step: the **Home Frame** provides navigation, the **Find Parking Frame** displays sample parking facilities, the **Parking Slot Frame** allows selection of an available slot, the **Booking Confirmation Frame** collects vehicle and duration details, and the **Parking Status Frame** is designed to display booking information.

Developing these frames helped apply Java Swing components, layout managers, and button actions in a practical project. The application also checks user input before continuing with a booking and calculates a fee based on the entered parking duration. The frames are arranged in a clear sequence, making the prototype straightforward to use and understand.

The project's main purpose was to build and connect the application screens. The parking facilities, prices, available-slot counts, and occupied slots are sample values entered in the program. The location search currently displays a message rather than filtering the facilities, and slot availability does not update after a booking. These limitations show where the application could be improved.

A separate **DBConnection** class provides the connection to MySQL. The booking and status frames use it to attempt to save and retrieve booking information.

In future development, the application could include more parking facilities, a location search that filters results, updated slot availability, rates that vary by facility, booking cancellation, and clearer booking-status results. Overall, the project provided practical experience in designing Java application frames and connecting a user interface to a database.

REFFERENCE

1. ***Oracle. Java Documentation. Available at: https://docs.oracle.com/en/java/***
2. ***Oracle. Java Swing Documentation. Available at: https://docs.oracle.com/javase/tutorial/uiswing/***
3. ***Oracle. MySQL 9.7 Reference Manual: MySQL Command Options. Available at: https://dev.mysql.com/doc/refman/9.7/en/mysql-command-options.html***
4. ***OpenAI-ChatGPT (GPT-4). Available at: https://chatgpt.com/***
5. ***GitHub. Smart Parking Sysyem — Project Repository. Available at: https://github.com/navnitm/smart-parking***